PROJECT REPORT

Orpheus Audio Dashboard

A Dual-Architecture Speech & Audio Feature Extraction System combining a Java Swing desktop GUI with a Python FastAPI ML backend.

Category	Digital Signal Processing / AI Tooling
Backend	Python 3.11 · FastAPI · Uvicorn · Librosa · SciPy
Frontend	Java 17 · Swing · FlatLaf 3.4 · MigLayout
Transport	HTTP/JSON over localhost:9999
Distribution	Docker (multi-stage) · Maven Shade Fat JAR
Version	1.0

Project Team & Contributions

Team Member	Domain	Primary Contributions
Abhirup Sarkar	Integration Lead	Feature selection logic, cross-language JSON serialization, Execution state, and async SwingWorker orchestration. (Major Contribution)
Aayush Dey	Frontend	Input/Output interface components and basic file save dialogs. (Minor Contribution)
Toufik Mamud	Backend	Core DSP preprocessing algorithms and librosa feature extraction implementation. (Major Contribution)
Subhrojyoti Bala	Backend Lead	FastAPI server infrastructure, REST endpoints, ThreadPool execution management, data flow, and containerization. (Major Contribution)
Anwesha	Frontend	Assisted with GUI architecture, UI/UX design, and FlatLaf theming. (Minor Contribution)

Table of Contents

1. Project Overview
2. System Architecture
3. End-to-End Data Flow
4. DSP Preprocessing Pipeline
5. Feature Extraction Modules
6. Complete Feature Inventory
7. Java Desktop GUI
8. REST API Reference
9. Docker Deployment
10. Tech Stack Summary
11. Repository Structure

■ 1. Project Overview

Orpheus is a production-grade, dual-architecture audio analysis platform that automates the extraction of acoustic feature vectors from raw speech and audio recordings. It is designed for AI researchers, speech scientists, and data engineers who need structured, machine-learning-ready CSV datasets from large audio collections — without writing any DSP code themselves.

The system consists of two fully decoupled components that communicate exclusively over HTTP/JSON on localhost. This makes each layer independently upgradeable, testable, and deployable.

Java Desktop GUI

- Built with Java 17 Swing + FlatLaf Dark theme
- Drag-and-drop audio file ingestion
- Toggle-based preprocessing and feature controls
- Live progress bar via SwingWorker HTTP polling
- One-click CSV and ZIP download

Python FastAPI Backend

- FastAPI + Uvicorn REST server on port 9999
- Librosa / SciPy / NumPy DSP computation engine
- Registry-pattern plugin architecture
- ThreadPoolExecutor for non-blocking CPU work
- Auto-creates output directories and archives

Key Capabilities

- MFCC + Δ + $\Delta\Delta$ $\rightarrow$ 78 features per file
- Chroma STFT (12 pitch classes) $\rightarrow$ 24 features
- Spectral Centroid, Bandwidth, Roll-off, Contrast, Flatness $\rightarrow$ 22 features
- F0 Pitch + Jitter via probabilistic YIN (pyin) $\rightarrow$ 5 features
- RMS Energy + Shimmer + HNR (autocorrelation) $\rightarrow$ 6 features
- Zero-Crossing Rate $\rightarrow$ 2 features
- Tempo (BPM) + Duration $\rightarrow$ 2 features
- Butterworth 4th-order High-Pass Filter (SciPy)
- Spectral Gating Noise Reduction (noisereduce)
- Docker-ready deployment — no Python install needed on host

■ 2. System Architecture

Orpheus follows a strict **microservice-inspired separation of concerns**. The Java GUI is a pure presentation layer with no ML or DSP logic whatsoever. All computation is delegated to the Python server over HTTP. The server is stateless per-request (progress state lives in a shared dict updated by callbacks), making it straightforward to add concurrent job support in future.

```
■■■
```

Step	Description
1. File Ingestion	User selects .wav/.mp3 files in the Java InputSection. Paths are stored as absolute strings.
2. Config Assembly	Java assembles an ExtractionConfig POJO (audioFiles, preprocessingSettings, featureSelection) and serialises it to JSON via GSON.
3. POST /api/config	JSON is sent to FastAPI. Pydantic validates the payload and the pipeline is queued as a BackgroundTask to keep the event loop free.
4. Pipeline Execution	ExtractionService.run() runs in a ThreadPoolExecutor. For each file: resolve path → load (librosa) → run preprocessors in order → save processed WAV → run all active extractors → accumulate feature row.
5. Progress Reporting	A shared status dict is updated via status_callback(). Java SwingWorker polls GET /api/status every 500 ms and renders a live progress bar.
6. Export & Archive	All rows are written to output/features/features.csv. Processed WAVs and the CSV are bundled into output/archives/speech_features.zip.
7. Download	Java OutputSection triggers GET /api/download/csv or GET /api/download/zip. FastAPI returns a FileResponse which the GUI saves to disk.

■ 4. DSP Preprocessing Pipeline

All preprocessing steps are plugins conforming to the **BasePreprocessor** abstract class. Regardless of which steps the GUI enables, the execution order is always fixed and canonical to ensure deterministic, reproducible output.

FIXED ORDER

Fixed Execution Order: Resample → High-Pass Filter → Noise Reduction → Silence Removal → Normalisation

#	Step	Class	Algorithm / Library	Key Parameters
1	Resample	Resampler	librosa.resample	Supports 8kHz, 16kHz, 22.05kHz, 44.1kHz, 48kHz
2	High-Pass Filter	HighPassFilter	4th-order Butterworth via scipy.signal	Cutoff Hz (default 80 Hz)
3	Noise Reduction	NoiseReducer	Stationary spectral gating (noisereduce)	stationary=True
4	Silence Removal	SilenceRemover	librosa.effects.trim	top_db threshold
5	Normalise	Normalizer	dBFS peak normalisation (NumPy)	Target (default –1 dBFS)

High-Pass Filter Detail

A 4th-order Butterworth filter is designed with **scipy.signal.butter** and applied via **sosfilt** (second-order sections) to avoid numerical instability. The cutoff frequency is normalised to the Nyquist before design:

```
sos = butter(4, cutoff_hz / nyquist, btype='high', output='sos')
y_filtered = sosfilt(sos, y).astype(np.float32)
```

Noise Reduction Detail

Spectral gating estimates the noise floor from the quietest spectral regions and subtracts it in the frequency domain. **stationary=True** assumes a consistent background noise profile:

```
y_clean = nr.reduce_noise(y=y, sr=sr, stationary=True)
```

NumPy Vector Telemetry & Array Transformations

A critical aspect of the backend pipeline is the continuous logging and transformation of the audio signal, represented as a 1D NumPy array. High-resolution telemetry monitors the array's shape, value bounds, and datatype synchronously at each step of processing to guarantee data integrity before feature extraction:

Array Semantics: The 1D NumPy array contains discrete, uncompressed floating-point samples of the audio waveform in the time domain. Each scalar value denotes the sound pressure amplitude at a specific point in time, bounded between roughly -1.0 and 1.0. The array length *N* represents the total number of frames, meaning the exact time coordinate of any given value can be derived via index / sample_rate = seconds.

Audio to Array Conversion: To generate this array, the backend uses librosa.load(), which relies on the soundfile library to decode the raw binary headers and byte streams of the .wav or .mp3 files. During this ingestion phase, multi-channel stereo audio is automatically downmixed to a mono signal by averaging the spatial channels, and the native 16-bit or 24-bit integer PCM amplitudes are scaled and cast down into normalized 32-bit floating-point numbers.

Processing Phase	NumPy Shape Change	Value Range / Mutation	Data Type
1. Ingestion (load)	Native shape → (N,)	Native (typically [-1.0, 1.0])	float32
2. Resampler	(N,) → (N_resampled,)	Interpolated variations	float32
3. High-Pass Filter	No shape change	Attenuates low-freq DC offset	float32
4. Noise Reduction	No shape change	Spectral variance reduced	float32
5. Silence Removal	(N_resampled,) → (N_trimmed,)	Unchanged (silent frames dropped)	float32
6. Normalizer	No shape change	Strictly scaled to [-1.0, 1.0]	float32

Example: Real-time Telemetry Trace

```
[TRACE] audio_file: speech_sample.wav
[TRACE] 1_Ingestion   | y.shape=(1323000,) | min=-0.83, max=0.84 | dtype=float32
[TRACE] 2_Resampling  | y.shape=(661500,)  | min=-0.84, max=0.85 | dtype=float32
[TRACE] 3_HighPass    | y.shape=(661500,)  | min=-0.82, max=0.84 | dtype=float32
[TRACE] 4_NoiseReduce | y.shape=(661500,)  | min=-0.79, max=0.81 | dtype=float32
[TRACE] 5_TrimSilence | y.shape=(610200,)  | min=-0.79, max=0.81 | dtype=float32
[TRACE] 6_Normalize   | y.shape=(610200,)  | min=-1.00, max=1.00 | dtype=float32
```

■ 5. Feature Extraction Modules

All extractors inherit from **BaseFeatureExtractor**, which mandates a **name** property, a **column_names()** list, and an **extract(y, sr)** method. The registry pattern in **ExtractionService** maps JSON key strings to extractor classes, enabling dynamic loading without modifying core orchestration logic.

MFCC Extractor (78 features)

Computes 13 Mel-Frequency Cepstral Coefficients, first-order deltas, and second-order delta-deltas via librosa.feature.mfcc and librosa.feature.delta. For each of the 39 coefficient tracks (13 × 3), mean and standard deviation are reported.

Columns: mfcc_1_mean … mfcc_13_std, mfcc_delta_1_mean … mfcc_delta2_13_std

Chroma Extractor (24 features)

Computes an STFT magnitude spectrogram then maps it to a 12-bin pitch-class (chroma) representation via librosa.feature.chroma_stft. Bins correspond to C, C#, D, D#, E, F, F#, G, G#, A, A#, B. Mean and std per bin.

Columns: chroma_1_mean … chroma_12_std

Spectral Extractor (22 features)

Extracts spectral centroid, bandwidth, roll-off (85th percentile), contrast across 7 sub-bands, and flatness. Mean + std reported for each descriptor.

Columns: spectral_centroid_mean/std, bandwidth_mean/std, rolloff_mean/std, contrast_1-7_mean/std, flatness_mean/std

Pitch (F0) Extractor (5 features)

Uses librosa.pyin (probabilistic YIN) to estimate frame-level F0. Search range C2–C7 (65–2093 Hz). Unvoiced frames (NaN) excluded. Jitter = mean|Δf0| / mean_f0 on voiced frames.

Columns: f0_mean, f0_std, f0_min, f0_max, jitter_relative

Energy Extractor (6 features)

RMS energy per frame via librosa.feature.rms. Shimmer = mean|ΔRMS| / mean_RMS. HNR estimated via normalised autocorrelation: HNR = 10·log10(r_max / (1 – r_max)) where r_max is the peak autocorrelation in the 75–500 Hz lag range.

Columns: rms_mean, rms_std, rms_min, rms_max, shimmer_relative, hnr_db

Zero-Crossing Rate (2 features)

Counts the rate of sign changes per frame via librosa.feature.zero_crossing_rate. High ZCR correlates with fricatives; low ZCR with voiced speech.

Columns: zcr_mean, zcr_std

Temporal Extractor (2 features)

Duration via librosa.get_duration. Tempo estimated in BPM via onset strength envelope and librosa.feature.tempo.

Columns: duration_sec, tempo_bpm

Execution Telemetry Screenshots

The exact terminal pipeline trace tracking NumPy array shapes and magnitudes:

```
D:\Orpheus\.venv\Scripts\pyt

[Load] ✔Loaded audio
[Load]   Path     : C:\Users\anite\OneDrive\Desktop\WhatsApp Ptt 2026-04-17 at 5.38.23 PM.ogg
[Load]   Array    : shape=(118045,), dtype=float32
[Load]   SR       : 16000 Hz
[Load]   Duration: 7.3778s  (118,045 samples)
[Load]   Amplitude: min=-0.945496, max=0.999969, mean=-0.000097, rms=0.117813
[Load]   y (first 30 samples):
         [ 0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00
  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  3.0517578e-05
  3.0517578e-05  3.0517578e-05 -3.0517578e-05  3.0517578e-05 -3.0517578e-05  3.0517578e-05 -3.0517578e-05
  3.0517578e-05  3.0517578e-05 -3.0517578e-05 -3.0517578e-05  3.0517578e-05 -3.0517578e-05
 -3.0517578e-05 -3.0517578e-05]
[  2%] [1/1] Preprocessing WhatsApp Ptt 2026-04-17 at 5.38.23 PM.ogg…

────────────────────────────────────────────────────────────
[Resampler] Input array  : shape=(118045,), dtype=float32
[Resampler] Sample rate  : 16000 Hz  →  target: 8000 Hz
[Resampler] Duration     : 7.3778s  (118,045 samples)
[Resampler] Amplitude    : min=-0.945496, max=0.999969, mean=-0.000097
[Resampler] y BEFORE (first 20 samples):
            [ 0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00
  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  3.0517578e-05
  3.0517578e-05  3.0517578e-05 -3.0517578e-05  3.0517578e-05 -3.0517578e-05  3.0517578e-05]
[Resampler] Output array : shape=(59023,), dtype=float32
[Resampler] New duration : 7.3779s  (59,023 samples)
[Resampler] Amplitude    : min=-0.948112, max=1.010871
[Resampler] y AFTER  (first 20 samples):
            [ 3.9585234e-06 -4.5836910e-06  5.2651548e-06 -6.8392267e-06  7.8888649e-06 -8.5794418e-06  1.4820293e-05
  2.4689224e-05  7.8584053e-06 -9.7476859e-06  1.3813591e-05  3.7184742e-06 -1.0274927e-05  1.7192811e-05
 -3.5534962e-05  1.2497376e-05  9.9965200e-06  1.0668093e-05 -3.0331952e-05 -2.8145093e-06]
[Resampler] ✔Resample complete  (16000 Hz → 8000 Hz)

────────────────────────────────────────────────────────────
[HighPassFilter] Input array    : shape=(59023,), dtype=float32
[HighPassFilter] Sample rate    : 8000 Hz
[HighPassFilter] Nyquist freq   : 4000.0 Hz
[HighPassFilter] Cutoff freq    : 100 Hz  (normalised: 0.02500)
[HighPassFilter] Filter order   : 4 (Butterworth)
[HighPassFilter] Amplitude pre  : min=-0.948112, max=1.010871, rms=0.117757
[HighPassFilter] y BEFORE (first 20 samples):
                 [ 3.9585234e-06 -4.5836910e-06  5.2651548e-06 -6.8392267e-06  7.8888649e-06 -8.5794418e-06  1.4820293e-05
  2.4689224e-05  7.8584053e-06 -9.7476859e-06  1.3813591e-05  3.7184742e-06 -1.0274927e-05  1.7192811e-05
 -3.5534962e-05  1.2497376e-05  9.9965200e-06  1.0668093e-05 -3.0331952e-05 -2.8145093e-06]
[HighPassFilter] SOS coefficients: shape=(2, 6)  (sections × 6)
[HighPassFilter]   SOS[0] = [0.90244446, -1.80488891, 0.90244446, 1.0, -1.85907627, 0.8648249]
[HighPassFilter] Output array   : shape=(59023,), dtype=float32
[HighPassFilter] Amplitude post : min=-1.016890, max=1.063837, rms=0.117428
[HighPassFilter] y AFTER  (first 20 samples):
                 [ 3.57234740e-06 -4.86960380e-06  4.94343567e-06 -6.24993572e-06  6.72908482e-06 -8.67260678e-06  1.41338485e-05
  2.01524235e-05  6.96657082e-07 -1.59692190e-05  7.16328937e-06 -3.09399434e-06 -1.56488278e-05  1.18313164e-05
 -3.84870627e-05  1.23404197e-05  7.84694283e-06  6.90564548e-06 -3.16823907e-05 -5.18142087e-07]
[HighPassFilter] ✔High-pass filter applied  (cutoff=100 Hz)

────────────────────────────────────────────────────────────
[NoiseReducer] Input array  : shape=(59023,), dtype=float32
[NoiseReducer] Sample rate  : 8000 Hz
```

```
D:\Orpheus\.venv\Scripts\pyt

[NoiseReducer] RMS power    : 0.117428  (before reduction)
[NoiseReducer] Amplitude    : min=-1.016890, max=1.063837
[NoiseReducer] Algorithm    : stationary spectral gating (noisereduce)
[NoiseReducer] y BEFORE (first 20 samples):
               [ 3.57234740e-06 -4.86960380e-06  4.94343567e-06 -6.24993572e-06  6.72908482e-06 -8.67260678e-06  1.41338485e-05
  2.01524235e-05  6.96657082e-07 -1.59692190e-05  7.16328937e-06 -3.09399434e-06 -1.56488278e-05  1.18313164e-05
 -3.84870627e-05  1.23404197e-05  7.84694283e-06  6.90564548e-06 -3.16823907e-05 -5.18142087e-07]
[NoiseReducer] Running spectral gate...
[NoiseReducer] Output array : shape=(59023,), dtype=float32
[NoiseReducer] RMS power    : 0.830627  (after reduction)
[NoiseReducer] Noise change : -11.67 dB
[NoiseReducer] y AFTER  (first 20 samples):
               [ 7.9977596e-23 -5.9272855e-23 -1.0365938e-22 -2.1730347e-23 -2.2685153e-22  7.2616668e-22 -1.7657378e-22
 -1.0897718e-21  3.9141418e-22  7.8699334e-22 -7.7084983e-22  2.1891794e-22  9.2532072e-23 -4.6386672e-22
  1.3388948e-21 -6.1575668e-23 -1.2086738e-21 -1.3780186e-22  1.2787469e-21  4.9673741e-22]
[NoiseReducer] ✔Noise reduction complete

────────────────────────────────────────────────────────────
[SilenceRemover] Input array     : shape=(59023,), dtype=float32
[SilenceRemover] Sample rate     : 8000 Hz
[SilenceRemover] Duration before : 7.3779s  (59,023 samples)
[SilenceRemover] Amplitude       : min=-0.563985, max=0.574506
[SilenceRemover] Silence thresh  : top_db=30  (anything 30 dB below peak = silent)
[SilenceRemover] y BEFORE (first 20 samples):
                 [ 7.9977596e-23 -5.9272855e-23 -1.0365938e-22 -2.1730347e-23 -2.2685153e-22  7.2616668e-22 -1.7657378e-22
 -1.0897718e-21  3.9141418e-22  7.8699334e-22 -7.7084983e-22  2.1891794e-22  9.2532072e-23 -4.6386672e-22
  1.3388948e-21 -6.1575668e-23 -1.2086738e-21 -1.3780186e-22  1.2787469e-21  4.9673741e-22]
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
[SilenceRemover] Non-silent intervals found: 5
[SilenceRemover]   [1] samples 1,536-5,632  (0.192s - 0.704s, duration=0.512s)
[SilenceRemover]   [2] samples 20,992-27,648  (2.624s - 3.456s, duration=0.832s)
[SilenceRemover]   [3] samples 29,696-46,880  (3.712s - 5.760s, duration=2.048s)
[SilenceRemover]   [4] samples 47,104-52,736  (5.888s - 6.592s, duration=0.704s)
[SilenceRemover]   [5] samples 55,296-59,023  (6.912s - 7.378s, duration=0.466s)
[SilenceRemover] Output array    : shape=(36500,), dtype=float32
[SilenceRemover] Duration after  : 4.5625s  (36,500 samples)
[SilenceRemover] Silence removed : 2.8150s  (38.2% of original)
[SilenceRemover] y AFTER  (first 20 samples):
                 [ 1.2846976e-06  1.0824992e-05  2.0348860e-05  2.9598339e-05  3.8407801e-05  4.6838275e-05  5.4759796e-05
  6.2284403e-05  6.9087029e-05  7.5249045e-05  8.0266851e-05  8.3588777e-05  8.4608269e-05  8.2432111e-05
  7.6259072e-05  6.5171247e-05  4.8558362e-05  2.6011790e-05 -2.3450666e-06 -3.6078716e-05]
[SilenceRemover] ✔Silence removal complete

────────────────────────────────────────────────────────────
[Normalizer] Input array   : shape=(36500,), dtype=float32
[Normalizer] Sample rate   : 8000 Hz
[Normalizer] Amplitude     : min=-0.563985, max=0.574506
[Normalizer] Peak sample   : 0.574506  (-4.81 dBFS)
```

```
D:\Orpheus\.venv\Scripts\pyt

[Normalizer] Target level  : -1.00 dBFS
[Normalizer] y BEFORE (first 20 samples):
             [ 1.2846976e-06  1.0824992e-05  2.0348860e-05  2.9598339e-05  3.8407801e-05  4.6838275e-05  5.4759796e-05
  6.2284403e-05  6.9807029e-05  7.5249045e-05  8.0266851e-05  8.3588777e-05  8.4608269e-05  8.2432111e-05
  7.0259072e-05  6.5171247e-05  4.8558362e-05  2.6811790e-05 -2.3450666e-06 -3.6078716e-05]
[Normalizer] Target linear : 0.891251
[Normalizer] Gain applied  : ×1.551334  (+3.81 dB)
[Normalizer] Output array  : shape=(36500,), dtype=float32
[Normalizer] New peak      : 0.891251  (-1.00 dBFS)
[Normalizer] y AFTER  (first 20 samples):
             [ 1.9929987e-06  1.6793177e-05  3.1567877e-05  4.5916985e-05  5.9583226e-05  7.2661881e-05  8.4958726e-05
  9.6499706e-05  1.0717785e-04  1.1673639e-04  1.2452868e-04  1.2967411e-04  1.3125567e-04  1.2787972e-04
  1.1838328e-04  1.0110236e-04  7.5330238e-05  4.8352973e-05 -3.6379813e-06 -5.5978136e-05]
[Normalizer] ✔Normalization complete  (target=-1.0 dBFS)
[  5%] [1/1] Extracting features from WhatsApp Ptt 2026-04-17 at 5.38.23 PM.ogg…

────────────────────────────────────────────────────────────
[MFCC] Input array   : shape=(36500,), dtype=float32, sr=8000 Hz
[MFCC] n_mfcc        : 13  →  computing 13 + delta + delta²
[MFCC] MFCC matrix   : shape=(13, 72)  (coefficients × frames)
[MFCC] Value range   : min=-839.7675, max=193.9973
[MFCC] Full MFCC matrix (13 coefficients × first 20 frames):
[[-488.82776  -312.8456   -257.22794  ... -484.67715  -468.50952  -482.6115  ]
 [ 125.45086   157.9447   153.04527  ...  -19.987855   -3.8294846   19.3093  ]
 [  46.62834    39.8832    52.017113 ...    0.8955991  28.722586    38.688683 ]
 ...
 [ -22.746992  -30.628244  -25.56736  ...    2.2886426  -1.6383662  -1.6842053]
 [ -22.852688  -16.448117  -12.987848 ...   15.658839    5.36285    1.8420913]
 [ -33.98259   -36.38063   -36.579674 ...   -2.9189656  -9.46167   -13.286086 ]]
[MFCC] Delta matrix  : shape=(13, 72), range=[-61.1515, 54.9690]
[MFCC] Delta (first 20 frames):
[[-15.96532   -15.96532   -15.96532  ... -12.031313    1.1444103  12.98581  ]
 [ -3.0969682  -3.0969682  -3.0969682 ... -13.357694   -0.35880718 13.518553 ]
 [ -1.6642351  -1.6642351  -1.6642351 ...  24.694715   19.56587    11.644776 ]
 ...
 [  4.305294    4.305294    4.305294  ...  -1.878554   -3.690842   -2.7946534 ]
 [  1.5946131   1.5946131   1.5946131 ...  -1.0968356  -2.8538085  -3.844963  ]
 [  2.9972386   2.9972386   2.9972386 ...   1.5111121   0.03093073 -2.9182975 ]]
[MFCC] Delta² matrix : shape=(13, 72), range=[-40.6907, 34.3527]
[MFCC] Delta² (first 20 frames):
[[-16.579432  -16.579432  -16.579432  ...  18.597794    8.248674   -0.6164646 ]
 [  1.9106832   1.9106832   1.9106832 ...  17.542896   14.778331    9.252746  ]
 [ -9.131843   -9.131843   -9.131843  ...  -5.7377434  -2.6248587  -1.5482242 ]
 ...
 [ -2.8507957  -2.8507957  -2.8507957 ...  -0.75973105 -2.0288253  -0.9225382 ]
 [  5.0393395   5.0393395   5.0393395 ...  -1.3327242  -0.41319603  0.62597066]
 [  0.6424681   0.6424681   0.6424681 ...  -1.7788336  -1.4383218  -1.7999067 ]]
[MFCC] Computed 78 features  (mean+std for each of 39 tracks)
[MFCC] mfcc_1_mean=-347.2530, mfcc_1_std=120.1302, mfcc_2_mean=84.1808
[MFCC] ✔MFCC extraction complete

────────────────────────────────────────────────────────────
[Chroma] Input array   : shape=(36500,), dtype=float32, sr=8000 Hz
[Chroma] n_chroma      : 12  (C, C#, D, D#, E, F, F#, G, G#, A, A#, B)
[Chroma] STFT magnitude: shape=(1025, 72)  (freq_bins × frames)
[Chroma] STFT range    : min=0.000000, max=62.879411, mean=0.413441
```

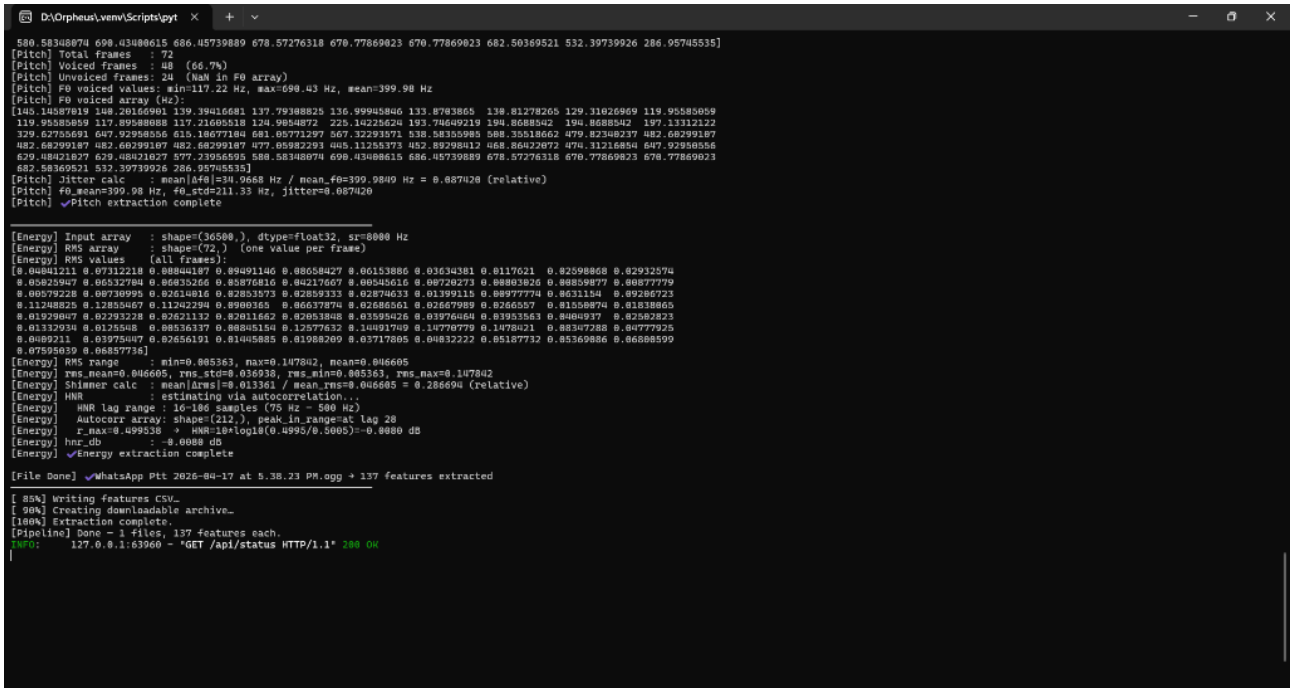
```
D:\Orpheus\.venv\Scripts\pyt

[Chroma] Chroma matrix : shape=(12, 72)  (12 pitch classes × frames)
[Chroma] Value range   : min=0.0055, max=1.0000
[Chroma] Full chroma matrix (12 rows = pitch classes, cols = first 20 frames):
[[0.34538868 0.48447763 0.5422438  ... 0.3077869  0.32466626 0.3173603 ]
 [0.7489151  1.         1.         ... 0.3631258  0.45192257 0.5048044 ]
 [1.         0.5966074  0.4103766  ... 0.17258377 0.21865514 0.28116113]
 ...
 [0.5226349  0.27956355 0.22189982 ... 0.5631637  0.48227537 0.2718616 ]
 [0.548367   0.18466863 0.16694716 ... 1.         1.         0.7333515 ]
 [0.38958927 0.16418581 0.16383722 ... 0.49868737 0.6252845  1.        ]]
[Chroma] Row means     : [0.41680000858386885, 0.49219998717388044, 0.45190088534857617, 0.43939998745918274, 0.40869998931884766, 0.4442000091075897, 0.3781800877724457, 0.3589000185857849, 0.37888004720687
87, 0.42689999938811117, 0.48399999737739563, 0.43458000882148741]  (C C# D D# E F F# G G# A A# B)
[Chroma] Computed 24 features  (mean+std per pitch class)
[Chroma]  ✔Chroma extraction complete

────────────────────────────────────────────────────────────
[ZCR] Input array  : shape=(36500,), dtype=float32, sr=8000 Hz
[ZCR] Duration     : 4.5625s
[ZCR] ZCR array    : shape=(72,)  (one value per frame)
[ZCR] ZCR values   (all frames):
[0.03369141 0.0625   0.09082031 0.10888672 0.12011719 0.11865234 0.12158203 0.11767578 0.12109375 0.11914062
 0.1328125  0.1796875  0.19042969 0.17919922 0.23339844 0.27929688 0.34814453 0.42333984 0.37597656 0.32128906
 0.24365234 0.17138672 0.13720703 0.10009766 0.10839844 0.13183594 0.15478516 0.16796875 0.15966797 0.1640625
 0.16218938 0.14892578 0.15722656 0.14550781 0.13574219 0.13818359 0.15820312 0.23095703 0.23730469 0.234375
 0.20458984 0.14990234 0.19921875 0.24121094 0.23681641 0.23876953 0.28361328 0.17333984 0.1875     0.18896484
 0.19189453 0.17431641 0.14941406 0.13183594 0.19140625 0.27246094 0.34082031 0.37255859 0.31201172 0.24568547
 0.19433594 0.16583906 0.23388672 0.21289062 0.21875    0.22412109 0.18896484 0.28857422 0.32275391 0.33105469
 0.25927734 0.14288984]
[ZCR] Value range  : min=0.033691, max=0.423340
[ZCR] zcr_mean     : 0.196574  (avg sign-changes per frame)
[ZCR] zcr_std      : 0.077181
[ZCR] ✔ZCR extraction complete

────────────────────────────────────────────────────────────
[Spectral] Input array   : shape=(36500,), dtype=float32, sr=8000 Hz
[Spectral] Centroid      : array=(72,), mean=962.89 Hz, std=480.66 Hz
[Spectral] Bandwidth     : array=(72,), mean=570.69 Hz, std=193.82 Hz
[Spectral] Roll-off@85%  : array=(72,), mean=1393.01 Hz, std=658.76 Hz
[spectral] Error on WhatsApp Ptt 2026-04-17 at 5.38.23 PM.ogg: Frequency band exceeds Nyquist. Reduce either fmin or n_bands.

────────────────────────────────────────────────────────────
[Pitch] Input array    : shape=(36500,), dtype=float32, sr=8000 Hz
[Pitch] Algorithm      : probabilistic YIN (pyin)
[Pitch] F0 search range: 65.41 Hz (C2) – 2093.00 Hz (C7)
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
[Pitch] F0 array       : shape=(72,)  (one value per frame)
[Pitch] F0 raw array   (NaN = unvoiced frame):
[145.14587019 148.20160901 139.39416601 137.79308825 136.99945846 133.8703865  130.81278265 129.31026969 119.95585059
 119.95585059 117.89508088 117.21605518          nan          nan          nan          nan          nan          nan
          nan          nan 124.9854872  225.14225624 193.74649219 194.8688542  194.8688542  197.13312122 329.62755691
 647.92950556 615.10677104 601.85771297 567.32293571 538.58355905 508.35518662 479.82348237 482.68299107 482.68299107
 482.68299107 482.68299107 477.85982293 445.11253373 452.89298412 468.86422872 474.31216054          nan          nan
          nan          nan          nan          nan          nan          nan          nan          nan          nan
          nan          nan          nan          nan          nan          nan 647.92950556 629.48421027 629.48421027 577.23956595
 580.58348874 698.43488615 686.45739889 678.57276318 670.77869023 670.77869023 682.50369521 532.39739926 286.95745535]
[Pitch] Total frames   : 72
```



```
D:\Orpheus\.venv\Scripts\pyt
580.58348074 690.43408615 686.45739889 678.57276318 670.77869023 670.77869023 682.50369521 532.39739926 286.95745535]
[Pitch] Total frames    : 72
[Pitch] Voiced frames   : 48  (66.7%)
[Pitch] Unvoiced frames: 24  (NaN in F0 array)
[Pitch] F0 voiced values: min=117.22 Hz, max=690.43 Hz, mean=399.98 Hz
[Pitch] F0 voiced array (Hz):
[145.14587819 148.20166901 139.39416681 137.79388825 136.99945846 133.8703865  138.81278265 129.31026969 119.95585059
 119.95585059 117.89588088 117.21605518 124.9854872  225.14225624 193.74649219 194.8688542  194.8688542  197.13312122
 329.62755659 647.92958556 615.18677184 601.05771297 567.32293571 538.58355985 588.35518662 479.82348237 482.68299187
 482.68299187 482.68299187 482.68299187 477.05982293 445.11255373 452.89298412 468.86422072 474.31216854 647.92958556
 629.48421027 629.48421027 577.23956595 580.58348074 690.43408615 686.45739889 678.57276318 670.77869023 670.77869023
 682.50369521 532.39739926 286.95745535]
[Pitch] Jitter calc     : mean|Δf0|=34.9668 Hz / mean_f0=399.9849 Hz = 0.087420 (relative)
[Pitch] f0_mean=399.98 Hz, f0_std=211.33 Hz, jitter=0.087420
[Pitch] ✔Pitch extraction complete
────────────────────────────────────────────────────────────
[Energy] Input array    : shape=(36500,), dtype=float32, sr=8000 Hz
[Energy] RMS array      : shape=(72,)  (one value per frame)
[Energy] RMS values     (all frames):
[0.04841211 0.07312218 0.08844187 0.09491146 0.08658427 0.06153886 0.03634381 0.0117621  0.02598868 0.02932574
 0.05825947 0.06532784 0.06835266 0.05876816 0.04217667 0.00545616 0.00720273 0.00803020 0.00859877 0.00877779
 0.00579228 0.00730995 0.02614016 0.02853573 0.02859333 0.02874633 0.01399115 0.00977774 0.00311154 0.09206723
 0.11248825 0.12855467 0.11242294 0.0900365  0.06637874 0.02686561 0.02667989 0.0266557  0.01550874 0.01838865
 0.01929047 0.02293228 0.02621132 0.02811662 0.02053848 0.03595426 0.03976464 0.03953563 0.0404937  0.02502823
 0.01332934 0.0125548  0.00536337 0.00845154 0.12577632 0.14491749 0.14778779 0.1478421  0.08347288 0.04777925
 0.0489211  0.03975447 0.02656191 0.01445885 0.01988209 0.03717805 0.04832222 0.05187732 0.05369886 0.06800599
 0.07595839 0.06857736]
[Energy] RMS range      : min=0.005363, max=0.147842, mean=0.046605
[Energy] rms_mean=0.046605, rms_std=0.036938, rms_min=0.005363, rms_max=0.147842
[Energy] Shimmer calc   : mean|Δrms|=0.013361 / mean_rms=0.046605 = 0.286694 (relative)
[Energy] HNR            : estimating via autocorrelation...
[Energy]   HNR lag range : 16-106 samples (75 Hz - 500 Hz)
[Energy]   Autocorr array: shape=(212,), peak_in_range=at lag 28
[Energy]   r_max=0.499538  →  HNR=10*log10(0.4995/0.5005)=-0.0080 dB
[Energy] hnr_db         : -0.0080 dB
[Energy] ✔Energy extraction complete

[File Done] ✔WhatsApp Ptt 2026-04-17 at 5.38.23 PM.ogg → 137 features extracted
────────────────────────────────────────────────────────────
[ 85%] Writing features CSV…
[ 90%] Creating downloadable archive…
[100%] Extraction complete.
[Pipeline] Done — 1 files, 137 features each.
INFO:     127.0.0.1:63960 - "GET /api/status HTTP/1.1" 200 OK
```

■ 6. Complete Feature Inventory

TOTAL

When all extractors are enabled: 78 + 24 + 22 + 5 + 6 + 2 + 2 = 139 scalar features per audio file (plus filename column).

Module	Feature Group	Count	Description
MFCC	mfcc_1…13 (mean + std)	26	Mel-Frequency Cepstral Coefficients
MFCC	mfcc_delta_1…13 (mean + std)	26	First-order temporal derivatives
MFCC	mfcc_delta2_1…13 (mean + std)	26	Second-order temporal derivatives
Chroma	chroma_1…12 (mean + std)	24	Pitch-class energy per semitone
Spectral	centroid, bandwidth, rolloff	6	Core spectral shape descriptors
Spectral	contrast_1…7 (mean + std)	14	Spectral contrast per sub-band
Spectral	flatness (mean + std)	2	Tonality vs. noise-likeness ratio
Pitch	f0_mean, f0_std, f0_min, f0_max	4	Fundamental frequency statistics
Pitch	jitter_relative	1	Pitch perturbation quotient
Energy	rms_mean, rms_std, rms_min, rms_max	4	Root-Mean-Square energy stats
Energy	shimmer_relative	1	Amplitude perturbation quotient
Energy	hnr_db	1	Harmonics-to-Noise Ratio (dB)
ZCR	zcr_mean, zcr_std	2	Zero-Crossing Rate statistics
Temporal	duration_sec, tempo_bpm	2	Duration and estimated rhythm
TOTAL		139	Scalar features per audio file

■ 7. Java Desktop GUI

The GUI is a standard Java Swing application themed with **FlatLaf 3.4** (Mac Dark variant), laid out with **MigLayout 11.4.2**, and uses **SVG Salamander** for vector icon rendering. All I/O and HTTP calls execute off the Event Dispatch Thread via **SwingWorker** to prevent UI freezing.

UI Section	Class	Responsibility
Input Section	InputSection	File picker / drag-drop; assembles the audio file list
Preprocessing	PreprocessingSection	Toggle switches and sliders for each DSP step
Feature Selection	FeatureSelectionSection	Checkbox grid to enable/disable individual extractors
Execution	ExecutionSection	Run button, JProgressBar, SwingWorker HTTP request/poll logic
Output	OutputSection	Download CSV / Download ZIP buttons; file-save dialogs

Cross-Language Contract

The Java **ExtractionConfig** POJO mirrors the Python Pydantic model field-for-field. GSON serialises on the Java side; Pydantic deserialises and validates on arrival. No IDL or code-generation tool is required — the contract is maintained by keeping both models in sync manually, which is safe given the HTTP boundary is internal and versioned together with the release.

```
// Java (ExtractionConfig.java)
List         audioFiles;
Map  preprocessingSettings;
Map featureSelection;

# Python (main.py — Pydantic)
audioFiles:            List[str]
preprocessingSettings: Dict[str, Any]
featureSelection:      Dict[str, bool]
```

■ 8. REST API Reference

The Python backend exposes five HTTP endpoints served by Uvicorn on port 9999. CORS is fully open (allow_origins=["*"]) to permit the Java desktop client to connect from any localhost port.

Method	Path	Description	Response
GET	/	Health check	JSON: {"status":"ok","message":"..."}
POST	/api/config	Submit extraction job	{"status":"success","message":"Pipeline started"}
GET	/api/status	Poll pipeline progress	{"progress":%,"message":"...","is_complete":bool}
GET	/api/download/csv	Download features CSV	FileResponse (text/csv)
GET	/api/download/zip	Download archive ZIP	FileResponse (application/zip)

Concurrency Model

The extraction pipeline is CPU-bound (librosa, scipy, numba). To avoid blocking FastAPI's asyncio event loop it runs inside a **ThreadPoolExecutor(max_workers=2)** via **asyncio.loop.run_in_executor**. A **409 Conflict** is returned if a job is already running. Progress is communicated via a shared in-process status dict rather than a database, keeping the architecture simple and zero-dependency.

■ 9. Docker Deployment

Orpheus ships with a **multi-stage Dockerfile** and **docker-compose.yml** that containerise the Python backend. The Java GUI runs natively on the host and connects to the containerised server. No Python, pip, Maven, or librosa installation is needed on the target machine.

MINIMAL HOST REQUIREMENTS

Host Requirements: Docker Desktop (any platform) + Java 17+ JRE. Nothing else.

Stage	Base Image	What it does
java-builder	maven:3.9-eclipse-temurin-17	Compiles Java source and packages everything into a fat JAR via Maven Shade Plugin
runtime	python:3.11-slim	Installs libsndfile1, ffmpeg, openjdk-21-jre-headless; installs all Python packages; copies Python source and the compiled fat JAR

Quick Start Commands

```
# 1. Build and start the backend
docker-compose up --build

# 2. Copy the compiled GUI JAR (first time only)
docker cp orpheus-backend:/app/Java/SpeechDashboard.jar ./SpeechDashboard.jar

# 3. Run the Java GUI on host (connects to localhost:9999 automatically)
java -jar SpeechDashboard.jar

# Verify backend health
curl http://localhost:9999/
```

Host Path	Container Path	Purpose
./data/input/	/app/Python/input/	Drop .wav/.mp3 files here before running
./data/output/	/app/Python/output/	CSVs, processed audio, and ZIP archive appear here

■ 10. Tech Stack Summary

Layer	Technology	Version	Role
Desktop GUI	Java Swing	17	Application window and user controls
GUI Theme	FlatLaf	3.4	Mac Dark modern look-and-feel
GUI Layout	MigLayout	11.4.2	Responsive panel layout engine
GUI Icons	SVG Salamander	1.0	Resolution-independent vector icons
JSON (Java)	GSON	2.11.0	Java ↔ JSON serialisation
Build (Java)	Maven + Shade Plugin	3.9/3.5.1	Dependency management and fat JAR
API Server	FastAPI + Uvicorn	latest	Async REST server on port 9999
Validation	Pydantic	v2	Request model validation
Audio Loading	Librosa + SoundFile	0.11+	WAV/MP3 ingestion and DSP utilities
DSP Filters	SciPy	latest	Butterworth filter design and sosfilt
Noise Reduction	NoiseReduce	3.x	Stationary spectral gating
Numerics	NumPy	2.x	Array operations throughout the pipeline
Data Export	Python csv (stdlib)	—	CSV writing with dynamic column headers
Containerisation	Docker (multi-stage)	29.x	Reproducible zero-install deployment

■ 11. Repository Structure

```
Orpheus/
■■■ Java/                          # Desktop GUI (Maven project)
■   ■■■ pom.xml
■   ■■■ src/main/java/com/speech/
■       ■■■ App.java               # Entry point
■       ■■■ model/ExtractionConfig.java
■       ■■■ ui/
■       ■   ■■■ MainDashboard.java
■       ■   ■■■ components/AnimatedButton.java
■       ■   ■■■ sections/
■       ■       ■■■ InputSection.java
■       ■       ■■■ PreprocessingSection.java
■       ■       ■■■ FeatureSelectionSection.java
■       ■       ■■■ ExecutionSection.java
■       ■       ■■■ OutputSection.java
■       ■■■ util/ (FontHelper, SVGIconHelper)
■
■■■ Python/                        # ML/DSP Backend (FastAPI)
■   ■■■ main.py                    # FastAPI app + 5 endpoints
■   ■■■ requirements.txt
■   ■■■ core/  (base_extractor, base_preprocessor, base_exporter)
■   ■■■ preprocessing/
■   ■   ■■■ resampler.py  highpass_filter.py  noise_reducer.py
■   ■   ■■■ silence_remover.py  normalizer.py
■   ■■■ extractors/
■   ■   ■■■ mfcc_extractor.py    chroma_extractor.py
■   ■   ■■■ spectral_extractor.py  zcr_extractor.py
■   ■   ■■■ pitch_extractor.py   energy_extractor.py
■   ■   ■■■ temporal_extractor.py
■   ■■■ services/extraction_service.py  # Pipeline orchestrator
■   ■■■ exporters/csv_exporter.py
■   ■■■ output/ (processed/ features/ archives/)
■
■■■ Dockerfile                     # Multi-stage build
■■■ docker-compose.yml
■■■ .dockerignore
■■■ run.ps1  /  run.sh             # Dev launchers
■■■ README.md
```